Parcial 2: Estructuras de Datos

Profesor: Brayan Sebastian Yepes Garcia

Integrantes:

Juan Sebastian Rincon Pinzon
Diego Alejandro Rodriguez Sandoval
Miguel Angel Sanchez Sandoval

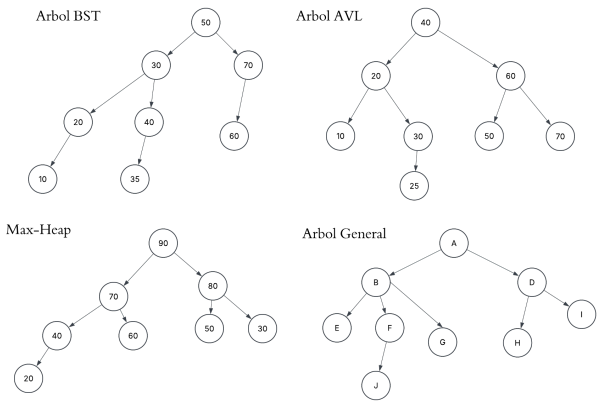
November 10, 2026

1 Problema elegido y estructuras de datos

1.1 Agentes de IA

Una tarea delegada a un agente; prioridad = urgencia de atención. No se requiere llamar a modelos de IA

1.2 Estructuras utilizadas:



1.3 Descripción breve de los árboles

- **BST:** árbol binario de búsqueda; los valores menores van a la izquierda y los mayores a la derecha.

- **AVL:** BST que se mantiene balanceado; las alturas de los subárboles de cada nodo difieren como máximo en uno.
- **Árbol general:** cada nodo puede tener cualquier cantidad de hijos.
- **Max-heap:** árbol binario completo donde cada padre es mayor o igual que sus hijos, por lo que el máximo queda en la raíz.

2 Tabla de complejidades

2.1 Complejidad temporal $T(n)$

Estructura	Búsqueda	Inserción	Eliminación
BST (promedio)	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$
BST (peor caso)	$O(n)$	$O(n)$	$O(n)$
AVL	$O(\log n)$	$O(\log n)$	$O(\log n)$

2.2 Complejidad espacial

Aspecto	BST	AVL	Max-Heap
Memoria total para n elementos	$O(n)$	$O(n)$	$O(n)$
Información adicional por nodo	No requiere datos de balance.	Un campo de altura o factor de balance ($O(1)$).	Sin metadatos por nodo; se representa en un arreglo.
Pila de recursión	$O(h)$; hasta $O(n)$ en el peor caso.	$O(\log n)$ en el peor caso.	$O(1)$ si se implementa de forma iterativa; $O(\log n)$ si se usa recursión.

2.3 Operaciones de Max-Heap

Operación	Descripción	Complejidad
max	Devuelve el elemento ubicado en la raíz, sin modificar el heap.	$O(1)$
$insert$	Añade un elemento y lo desplaza hacia arriba hasta restaurar la propiedad del heap.	$O(\log n)$
$extractMax$	Retira la raíz, coloca el último elemento en su lugar y lo desplaza hacia abajo.	$O(\log n)$
$BuscarPorId$	El heap solo ordena padre contra hijo, así que no hay forma de descartar mitades	$O(n)$
Eliminar un elemento	Hay que encontrarlo (O(n)) y luego arreglar el heap (O(log n)).	$O(n)$
Construir el Heap	La mayoría de los nodos están cerca de las hojas y bajan muy poco	$O(n)$
$heapSort$	Construir cuesta $O(n)$ y las n extracciones cuestan $O(logn)$ cada una	$O(nlogn)$

3 Analisis

3.1 Como evitar que una tarea cancelada sea atendida despues

Las cuatro estructuras comparten los mismos objetos `Registro`. `CANCELAR id` marca como `CANCELADO` las tareas pendientes del subárbol; las atendidas no cambian y el BST/AVL conservan el historial. Luego se reconstruye con `buildHeap` el Max-Heap usando solo tareas pendientes ($O(n)$), por lo que ninguna cancelada puede ser atendida por `SIGUIENTE`.

3.2 Comparación entre BST y AVL

3.2.1 Por qué insertar claves ordenadas perjudica al BST y no al AVL

Con claves ordenadas, el BST inserta cada valor al final y puede formar una cadena de altura $n-1$, haciendo sus operaciones $O(n)$. El AVL reequilibra el árbol con rotaciones cuando $|FB| = 2$ y mantiene altura $O(\log n)$; por ello, búsqueda, inserción y eliminación son $O(\log n)$ en el peor caso.

3.2.2 Comparación de alturas con 1..7 y 1..1000

Con hojas de altura 0 y árbol vacío de altura -1:

- 1..7**:** altura BST 6; altura AVL 2 (raíz 4, con hijos 2 y 6).
- 1..1000**:** altura BST 999; en la inserción ordenada, el AVL mantiene altura cercana a 9.

3.2.3 Balance, ventajas y limitaciones

En cada nodo AVL, las alturas de los subárboles difieren como máximo en uno; si se rompe esta condición, se aplican rotaciones simples o dobles (LL, RR, LR o RL). Esto evita la degradación del BST, aunque requiere actualizar la altura o el factor de balance.

4 Cancelación, errores encontrados y decisiones tomadas

4.1 Explicación del mecanismo de cancelación en cascada

Se busca el `id` en el Árbol General; si no existe, se informa el error. Si existe, un recorrido en profundidad cancela las tareas pendientes del subárbol, conserva las atendidas y reconstruye el heap con las restantes.

4.2 Errores encontrados durante el desarrollo y su solución

1. **Empates en el heap:** Se prioriza mayor urgencia y, en empate, menor ID.
2. **Heap alterado al reportar:** Se ordena una copia de los pendientes, no el arreglo original.
3. **Alturas del AVL:** En rotaciones dobles se actualizan primero los nodos que bajan.

4.3 Decisiones tomadas

1. Se usa el sucesor inorden al eliminar nodos con dos hijos.
2. BST y AVL conservan registros cancelados y atendidos para mantener el historial.
3. Los ID son únicos y cada tarea nueva se agrega bajo un padre existente, evitando ciclos.